

module. Now, let us look at how to load and unload this module in the kernel. Please note that you must have super-user privileges to load/unload kernel modules. To load a module, switch to the super-user mode and execute the *insmod* command, as shown below:

```
[root]# insmod hello.ko
```

insmod has done its job successfully. But where is the output? It is appended to the kernel's ring buffer. So let's verify it by executing the *dmesg* command:

```
[root]# dmesg
Hello, World !!!
```

We can also check whether our module is loaded or not. For this purpose, let's use the *lsmod* command:

```
[root]# lsmod | grep hello
hello                859  0
```

To unload the module from the kernel, just execute the *rmmod* command as shown below and check the output of the *dmesg* command. Now, *dmesg* shows the message from the clean-up function:

```
[root]# rmmod hello
```

```
[root]# dmesg
Hello, World !!!
Exiting ...
```

In this module, we have used a couple of macros, which provide information about the module. The *modinfo* command displays this information in a nicely formatted fashion:

```
[mickey]$ modinfo hello.ko
filename:      hello.ko
version:       1.0
description:   Hello world module.
author:        Narendra Kangralkar.
license:       GPL
srcversion:    144DCA60AA8E0CFC9899E3
depends:
vermagic:      2.6.32-358.el6.x86_64 SMP mod_unload
modversions
```

Finding the PID of a process

Let us write one more kernel module to find out the Process ID (PID) of the current process. The kernel stores all process related information in the *task_struct* structure, which is defined in the *<linux/sched.h>* header file. It provides a *current* variable, which is a pointer to the current process. To find out the PID of a current process, just print the value

of the *current->pid* variable. Given below is the complete working code (*pid.c*):

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/sched.h>

static int __init pid_init(void)
{
    printk(KERN_INFO "pid = %d\n", current->pid);

    return 0;
}

static void __exit pid_exit(void)
{
    /* Don't do anything */
}

module_init(pid_init);
module_exit(pid_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Narendra Kangralkar.");
MODULE_DESCRIPTION("Kernel module to find PID.");
MODULE_VERSION("1.0");
```

The *Makefile* is almost the same as the first *makefile*, with a minor change in the object file's name:

```
obj-m += pid.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Now compile and insert the module and check the output using the *dmesg* command:

```
[mickey]$ make

[root]# insmod pid.ko

[root]# dmesg
pid = 6730
```

A module that spans multiple files

So far we have explored how to compile a module from a single file. But in a large project, there are several source files for a single module and, sometimes, it is convenient to